

Verificación de una vulnerabilidad de inyección SQL (SQLi) en DVWA:

Paso 1: Preparación y acceso al entorno vulnerable

- Abre el navegador y navega a la URL local donde está desplegada la aplicación DVWA, usualmente <http://localhost/dvwa>.
- Inicia sesión con las credenciales estándar para entorno de pruebas ([admin](#) / [password](#)).
- En el menú de configuración, selecciona el nivel de seguridad [Low](#) dentro de la opción "DVWA Security" para desactivar mecanismos de protección y permitir la explotación básica de vulnerabilidades.

Este paso asegura que la aplicación no bloquee los intentos de inyección, emulando un entorno vulnerable para pruebas.

Paso 2: Navegación al módulo vulnerable de inyección SQL

- Desde el menú lateral de la aplicación, accede al módulo titulado "SQL Injection".
- Observa el formulario que solicita un identificador de usuario (User ID) y que al enviarlo devuelve información relacionada a ese ID.
- Entiende que la entrada del usuario es utilizada directamente en la construcción de una consulta SQL para recuperar información, y es aquí donde radica el vector de ataque.

Esta comprensión es clave para diseñar el payload que altere la lógica de la consulta SQL.

Paso 3: Ejecución de la prueba simple de inyección SQL

En el campo de texto para el ID de usuario, no ingreses un número válido sino el siguiente payload:

```
1' OR '1'='1
```

- Envía el formulario para que la aplicación procese la solicitud.

- Observa el resultado: si la aplicación devuelve múltiples registros de usuarios en lugar de uno solo, significa que la lógica de la consulta SQL ha sido manipulada exitosamente para que la condición sea siempre verdadera.

El payload inserta una condición lógica tautológica `'1'='1'` que anula la condición original y provoca que la base de datos retorne todos los registros.

Paso 4: Análisis detallado del resultado

- Identifica que el campo de entrada no realiza ninguna sanitización ni validación para impedir la inserción de caracteres especiales ni fragmentos de código SQL.
- Comprueba que la aplicación concatena directamente la entrada del usuario en la sentencia SQL sin utilizar parámetros o prepared statements, lo que permite la ejecución de código arbitrario dentro de la consulta.
- Reconoce que esta situación representa una vulnerabilidad crítica de inyección SQL (SQLi) que podría permitir a un atacante obtener acceso no autorizado a información sensible.

Esta etapa confirma la naturaleza y gravedad de la falla.

Paso 5: Documentación profesional del hallazgo

- Elabora un informe con:
 - **Título:** Ejemplo "Vulnerabilidad de Inyección SQL en módulo SQL Injection de DVWA".
 - **Descripción técnica:** Explica cómo la falta de validación y uso de concatenación directa de entradas permite alterar la consulta SQL.
 - **Evidencia:** Incluye comandos usados, capturas de pantalla o logs donde se evidencie la lista múltiple de usuarios obtenida.
 - **Evaluación del riesgo:** Describe la capacidad de un atacante para acceder a todos los datos de la tabla o incluso escalar la vulnerabilidad para ejecutar comandos SQL más dañinos.
 - **Recomendaciones:** Sugiere implementar consultas parametrizadas/prepared statements, sanitización y validación estricta de entradas, así como la configuración de entornos seguros y monitoreo de intentos de inyección.

- **Referencias:** OWASP, CWE-89 (SQL Injection), CVSS para valoración del riesgo.

Este paso es esencial para comunicar la vulnerabilidad a los responsables y facilitar su corrección.

Consideraciones técnicas para mitigación

- Implementar prepared statements o consultas parametrizadas que eviten la concatenación directa de cadenas con datos del usuario.
- Validar y sanear todas las entradas para impedir caracteres que puedan alterar la lógica SQL (como comillas simples, puntos y coma, etc.).
- Configurar el entorno de producción con un nivel de seguridad elevado, incluyendo monitoreo de intentos de inyección y bloqueo automático.
- Aplicar el principio de menor privilegio en la base de datos para limitar el daño en caso de explotación.
- Mantener registros (logs) detallados para detectar patrones de ataque.